



Факультет Систем Управления и Робототехники

## РАЗРАБОТКА ПРОГРАММНОЙ СИСТЕМЫ ДЛЯ ТОЧНОГО ДЕТЕКТИРОВАНИЯ МЕТОК НА КАЛИБРОВОЧНЫХ КАРТАХ ГАЛЬВАНOMETРИЧЕСКОГО СКАНАТОРА УСТАНОВКИ SLM ПЕЧАТИ

**Аннотация** – Работа посвящена разработке программной системы для автоматического детектирования меток на калибровочных картах гальванометрического сканатора установки SLM-печати. Выполнен анализ методов обработки изображений и локализации объектов. Разработаны модули предварительной обработки изображений, детектирования меток и визуализации результатов. Программное обеспечение реализовано на языке Python с использованием библиотеки OpenCV. Проведено исследование точности определения центров меток и оценка производительности системы. По результатам работы определена минимальная ошибка локализации центра метки.

**Ключевые слова:** SLM-печать, гальванометрический сканатор, калибровочная карта, компьютерное зрение, обработка изображений, детектирование меток, локализация объектов, OpenCV, Python.

**Выполнили**

Семенидо М.С<sup>1</sup>

<sup>1</sup>*R3380, 7enidomaks@gmail.com*

**Проверила**

Анастасия А. Крылова

# Содержание

|          |                                                                                                       |           |
|----------|-------------------------------------------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Введение</b>                                                                                       | <b>3</b>  |
| <b>2</b> | <b>Анализ предметной области</b>                                                                      | <b>4</b>  |
| 2.1      | Калибровка гальванометрических сканаторов . . . . .                                                   | 4         |
| 2.2      | Методы обработки изображений и обнаружения маркеров . . . . .                                         | 4         |
| 2.2.1    | Нормализация яркости и повышение контраста . . . . .                                                  | 4         |
| 2.2.2    | Морфологическое преобразование Top-Hat . . . . .                                                      | 4         |
| 2.2.3    | Бинаризация изображения . . . . .                                                                     | 5         |
| 2.2.4    | Морфологическая фильтрация . . . . .                                                                  | 5         |
| 2.2.5    | Определение центров маркеров . . . . .                                                                | 6         |
| 2.2.6    | Анализ локальной структуры изображения . . . . .                                                      | 6         |
| 2.2.7    | Выводы . . . . .                                                                                      | 7         |
| <b>3</b> | <b>Разработка алгоритма восстановления сеточной структуры и определе-<br/>ния координатных систем</b> | <b>7</b>  |
| 3.1      | Постановка задачи . . . . .                                                                           | 7         |
| 3.2      | Особенности калибровочной карты . . . . .                                                             | 7         |
| 3.3      | Перспективные искажения изображения . . . . .                                                         | 8         |
| 3.4      | Восстановление регулярной структуры сетки . . . . .                                                   | 9         |
| 3.5      | Определение специальных маркеров . . . . .                                                            | 9         |
| 3.6      | Определение начала координат . . . . .                                                                | 10        |
| 3.7      | Определение направления осей . . . . .                                                                | 10        |
| 3.8      | Формирование системы координат . . . . .                                                              | 11        |
| 3.9      | Визуализация результатов . . . . .                                                                    | 11        |
| 3.10     | Выводы . . . . .                                                                                      | 12        |
| 3.11     | Пользовательский интерфейс и конфигурирование системы . . . . .                                       | 12        |
| 3.11.1   | Консольный интерфейс (CLI) . . . . .                                                                  | 12        |
| 3.11.2   | Графический интерфейс (GUI) . . . . .                                                                 | 13        |
| <b>4</b> | <b>Экспериментальные исследования</b>                                                                 | <b>13</b> |
| 4.1      | Описание программного комплекса . . . . .                                                             | 13        |
| 4.2      | Методика и результаты экспериментов . . . . .                                                         | 14        |
| 4.3      | Оценка производительности . . . . .                                                                   | 14        |
| 4.4      | Выводы по экспериментам . . . . .                                                                     | 14        |
| <b>5</b> | <b>Заключение</b>                                                                                     | <b>15</b> |
| <b>6</b> | <b>Приложение</b>                                                                                     | <b>15</b> |

# 1 Введение

Технология селективного лазерного сплавления (Selective Laser Melting, SLM) является одним из наиболее распространённых методов аддитивного производства металлических изделий сложной геометрической формы. Качество получаемых деталей во многом определяется точностью позиционирования лазерного луча в рабочей области установки.

Для обеспечения высокой точности изготовления изделий используются гальванометрические сканаторы, осуществляющие быстрое отклонение лазерного луча по заданной траектории. Однако в процессе эксплуатации возможно возникновение геометрических искажений, вызванных особенностями оптической системы, погрешностями установки элементов и изменением параметров оборудования. Для компенсации подобных искажений требуется проведение регулярной калибровки сканирующей системы.

Одним из способов калибровки является использование специальных калибровочных карт, содержащих набор маркеров, расположенных в заранее известных позициях. Анализ изображений таких карт позволяет определить отклонения фактических координат маркеров от их эталонных значений и сформировать корректирующие преобразования для системы управления сканатором.

В связи с этим возникает задача автоматического обнаружения маркеров на изображениях калибровочных карт и определения их координат с высокой точностью. Решение данной задачи требует применения методов компьютерного зрения и цифровой обработки изображений, обеспечивающих устойчивую работу алгоритма при наличии шумов, неравномерного освещения, перспективных искажений и других факторов, влияющих на качество изображения.

Целью данной работы является разработка программной системы автоматического детектирования меток на калибровочных картах гальванометрического сканатора установки SLM-печати.

Для достижения поставленной цели необходимо решить следующие задачи:

- провести анализ существующих методов обработки изображений и локализации объектов;
- разработать алгоритм предварительной обработки изображений калибровочных карт;
- реализовать алгоритм обнаружения центров маркеров;
- разработать метод восстановления регулярной структуры калибровочной сетки;
- реализовать алгоритм автоматического определения координатных систем по специальным меткам;
- провести экспериментальное исследование точности локализации маркеров;
- оценить производительность разработанного программного обеспечения.

Объектом исследования является процесс обработки изображений калибровочных карт гальванометрического сканатора.

Предметом исследования являются алгоритмы компьютерного зрения и методы цифровой обработки изображений, применяемые для автоматического обнаружения и локализации маркеров.

Практическая значимость работы заключается в разработке программного обеспечения, позволяющего автоматизировать процесс анализа калибровочных карт и повысить точность настройки гальванометрических сканаторов установок SLM-печати.

## 2 Анализ предметной области

### 2.1 Калибровка гальванометрических сканаторов

Гальванометрический сканатор представляет собой систему быстрого управления положением лазерного луча посредством изменения углов поворота зеркал. В установках SLM-печати такие системы используются для формирования траектории движения лазера по поверхности порошкового слоя.

Даже незначительные ошибки позиционирования луча могут приводить к геометрическим отклонениям изготавливаемых изделий. Поэтому важным этапом эксплуатации оборудования является калибровка сканирующей системы.

Для выполнения калибровки используются специальные тестовые шаблоны или калибровочные карты, содержащие набор маркеров с заранее известными координатами. Сравнение измеренных координат маркеров с их эталонными значениями позволяет определить величину и характер возникающих искажений.

Автоматизация процесса обработки калибровочных карт позволяет существенно сократить время настройки оборудования и уменьшить влияние человеческого фактора на результаты измерений.

### 2.2 Методы обработки изображений и обнаружения маркеров

Одним из ключевых этапов анализа калибровочной карты является выделение маркеров на изображении и определение их координат. Качество выполнения данной процедуры непосредственно влияет на точность последующих вычислений и корректность результатов калибровки.

На практике изображения калибровочных карт могут содержать шумы, неравномерное освещение, блики, изменения контрастности и перспективные искажения. Поэтому перед выполнением поиска маркеров необходимо провести предварительную обработку изображения.

#### 2.2.1 Нормализация яркости и повышение контраста

После загрузки изображения производится его преобразование в полутоновое представление. Использование изображений в оттенках серого позволяет уменьшить объём обрабатываемых данных и упростить дальнейшие вычисления.

Для повышения качества выделения объектов применяется адаптивное выравнивание гистограммы с ограничением контраста (CLANE — Contrast Limited Adaptive Histogram Equalization).

В отличие от классического выравнивания гистограммы, метод CLANE выполняет обработку локальных областей изображения, что позволяет эффективно компенсировать неравномерное освещение и улучшить видимость деталей.

В результате повышается контраст между маркерами и фоном, что существенно упрощает последующую бинаризацию изображения.

#### 2.2.2 Морфологическое преобразование Top-Hat

Для выделения светлых объектов на тёмном фоне используется морфологическое преобразование Top-Hat.

Данная операция определяется следующим выражением:

$$I_{TH} = I - (I \circ B),$$

где:

- $I$  — исходное изображение;
- $B$  — структурный элемент;
- $\circ$  — операция морфологического открытия.

Преобразование Тор-Нат позволяет выделить локальные светлые области изображения и подавить медленно изменяющуюся составляющую фона.

В рассматриваемой задаче данный метод позволяет эффективно выделять маркеры калибровочной карты независимо от общего уровня освещённости изображения.

### 2.2.3 Бинаризация изображения

После выполнения контрастирования и морфологической обработки производится преобразование изображения в бинарный вид.

Для выбора порогового значения используется метод Отсу, автоматически определяющий оптимальную границу разделения яркостных классов.

Пусть изображение содержит два класса пикселей:

- фон;
- объекты.

Метод Отсу выбирает такой порог  $T$ , при котором межклассовая дисперсия достигает максимального значения.

Использование данного подхода позволяет отказаться от ручной настройки пороговых параметров и обеспечивает устойчивую работу алгоритма на различных изображениях.

### 2.2.4 Морфологическая фильтрация

После бинаризации на изображении могут присутствовать шумовые объекты и мелкие артефакты.

Для их удаления используются морфологические операции расширения и открытия.

Операция расширения позволяет устранить разрывы внутри объектов и повысить связность контуров.

Операция открытия применяется для удаления мелких шумовых областей, не относящихся к маркерам.

Дополнительно выполняется фильтрация контуров по следующим признакам:

- площадь объекта;
- отношение ширины к высоте;
- степень заполненности ограничивающего прямоугольника.

Это позволяет исключить ложные объекты и оставить только потенциальные маркеры калибровочной карты.

### 2.2.5 Определение центров маркеров

После формирования бинарной маски выполняется поиск контуров обнаруженных объектов.

Для каждого контура вычисляются геометрические моменты:

$$M_{pq} = \sum_x \sum_y x^p y^q I(x, y).$$

На их основе определяется положение центра масс объекта:

$$c_x = \frac{M_{10}}{M_{00}},$$

$$c_y = \frac{M_{01}}{M_{00}}.$$

Полученные координаты используются в качестве первоначальной оценки положения центра маркера.

Для повышения точности локализации применяется алгоритм субпиксельного уточнения, реализованный в библиотеке OpenCV функцией `cornerSubPix()`.

Данный метод позволяет определять координаты характерных точек с точностью значительно выше одного пикселя, что особенно важно при выполнении калибровочных измерений.

### 2.2.6 Анализ локальной структуры изображения

Не каждый найденный контур соответствует реальному узлу калибровочной сетки. Для дополнительной проверки используется анализ локальной структуры изображения в окрестности найденной точки.

В небольшой области вокруг центра вычисляются градиенты изображения по горизонтальному и вертикальному направлениям:

$$\frac{\partial I}{\partial x}, \quad \frac{\partial I}{\partial y}.$$

На основе полученных производных формируется матрица второго момента:

$$H = \begin{bmatrix} \sum dx^2 & \sum dx dy \\ \sum dx dy & \sum dy^2 \end{bmatrix}.$$

После этого вычисляются собственные значения матрицы:

$$\lambda_1, \lambda_2.$$

Если оба собственных значения имеют сопоставимые величины, то рассматриваемая область содержит пересечение линий или характерную угловую структуру.

Для количественной оценки используется отношение

$$R = \frac{\lambda_{min}}{\lambda_{max}}.$$

Если значение коэффициента превышает заданный порог, точка считается корректным узлом калибровочной сетки.

Применение данного подхода позволяет существенно уменьшить количество ложных срабатываний и повысить устойчивость работы алгоритма при наличии шумов и дефектов изображения.

### 2.2.7 Выводы

В результате анализа были выбраны методы предварительной обработки изображений, обеспечивающие устойчивое обнаружение маркеров на калибровочных картах. Использование CLANE, морфологического преобразования Тор-Нат, бинаризации Отсу и субпиксельной локализации позволяет повысить точность определения координат маркеров и обеспечить надёжную работу системы в условиях реальных производственных изображений.

## 3 Разработка алгоритма восстановления сеточной структуры и определения координатных систем

### 3.1 Постановка задачи

После обнаружения центров маркеров необходимо определить их взаимное расположение на калибровочной карте.

В идеальном случае все маркеры образуют регулярную прямоугольную сетку с фиксированным шагом между соседними элементами. Однако реальные изображения содержат перспективные искажения, возникающие вследствие наклона камеры относительно плоскости карты.

Кроме того, часть маркеров может быть обнаружена с погрешностью либо отсутствовать вследствие дефектов изображения.

Поэтому следующей задачей является восстановление регулярной структуры калибровочной сетки и определение координатных систем, заданных специальными маркерами.

Для решения данной задачи необходимо:

- восстановить геометрическую структуру сетки;
- сопоставить каждой точке её положение в сетке;
- определить специальные маркеры;
- автоматически выделить начала координат;
- вычислить направления осей локальных систем координат.

### 3.2 Особенности калибровочной карты

Используемая калибровочная карта представляет собой регулярную прямоугольную сетку.

Пусть количество строк равно

$$N_r = 9,$$

а количество столбцов

$$N_c = 11.$$

Расстояние между соседними узлами составляет

$$d = 35; \text{ мм.}$$

Тогда каждая точка сетки может быть описана индексами:  $(i, j)$ .

где

$$i \in 0, N_r - 1$$

$$j \in 0, N_c - 1$$

Соответствующие физические координаты определяются выражением

$$X = j \cdot d,$$

$$Y = i \cdot d.$$

Таким образом формируется эталонная геометрическая модель калибровочной карты.

### 3.3 Перспективные искажения изображения

При съёмке калибровочной карты камера редко располагается строго перпендикулярно её поверхности.

Вследствие этого возникает перспективное преобразование.

Пусть точка плоскости имеет координаты

$$(x, y).$$

После проецирования на изображение её координаты определяются матрицей гомографии:

$$H \begin{bmatrix} x \\ y \\ 1 \end{bmatrix}, \quad s \begin{bmatrix} u \\ v \\ 1 \end{bmatrix}$$

где

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix}$$

является матрицей перспективного преобразования.

В результате расстояния между соседними маркерами на изображении становятся неодинаковыми, а прямоугольная сетка приобретает трапецидальную форму.

Без устранения данных искажений невозможно корректно определить положение точек внутри сеточной структуры.



### 3.4 Восстановление регулярной структуры сетки

После обнаружения центров маркеров выполняется процедура восстановления сеточной структуры.

Основная идея заключается в переходе от координат изображения к координатам регулярной сетки.

На основании обнаруженных точек строится геометрическая модель расположения маркеров.

Далее производится оценка параметров перспективного преобразования, позволяющего перевести изображение в нормализованную систему координат.

После выполнения коррекции каждая точка получает уникальный индекс

$$(row; col)$$

Например,

$$(0, 0)$$

соответствует левому верхнему узлу сетки,

а

$$(8, 10)$$

соответствует правому нижнему узлу.

После завершения восстановления каждая обнаруженная точка обладает следующими характеристиками:

- координатами на изображении;
- номером строки;
- номером столбца;
- физическими координатами в миллиметрах.

Таким образом формируется полная цифровая модель калибровочной карты.

### 3.5 Определение специальных маркеров

Для задания локальных координатных систем используются специальные маркеры, дополнительно выделенные окружностями.

После восстановления структуры сетки выполняется повторный анализ бинарного изображения.

На первом этапе находятся все внешние контуры.

Для каждого контура вычисляется площадь:

$$S = Area(contour).$$

Поскольку окружности имеют существенно большие размеры по сравнению с обычными маркерами, производится фильтрация по площади.

Для оставшихся контуров вычисляются геометрические моменты и координаты центра:

$$c_x = \frac{M_{10}}{M_{00}},$$

$$c_y = \frac{M_{01}}{M_{00}}.$$

Полученная точка не обязательно совпадает с центром узла сетки.

Поэтому выполняется поиск ближайшего узла среди восстановленных точек.

Для ускорения вычислений используется структура данных KD-Tree.

Пусть

$$P_c = (c_x, c_y)$$

является центром найденного контура.

Тогда определяется ближайшая точка сетки

$$P_g = \arg, \min_{p_i} ||P_c - p_i||.$$

Если расстояние

$$||P_c - P_g||$$

не превышает установленный порог, то данный узел считается специальным маркером.

В результате формируется множество отмеченных точек

$$M = m_1, m_2, \dots, m_k.$$

### 3.6 Определение начала координат

Каждый специальный маркер потенциально может являться началом локальной системы координат.

Пусть

$$O = (r_o, c_o)$$

обозначает текущий рассматриваемый узел.

Необходимо определить точки, задающие направления осей.

Для этого среди остальных специальных маркеров выполняется поиск соседнего узла:

$$|r - r_o| + |c - c_o| = 1.$$

Такое условие соответствует непосредственному соседству по строке или столбцу.

Найденная точка используется для задания направления оси X.

Если соответствующий сосед отсутствует, построение системы координат прекращается.

### 3.7 Определение направления осей

После нахождения точки оси X определяется её направление относительно начала координат.

Пусть

$$\Delta r = r_x - r_o,$$

$$\Delta c = c_x - c_o.$$

Тогда направление оси X описывается вектором

$$d_x = (\Delta r, \Delta c).$$

Для построения второй оси необходимо найти направление, ортогональное оси X. В дискретном случае используются два возможных направления:

$$d_y = (-d_x^y, d_x^x),$$

или

$$d_y = (d_x^y, -d_x^x).$$

После этого среди специальных маркеров выполняется поиск точки, расположенной на расстоянии двух шагов сетки вдоль найденного направления.

Если такая точка существует, она принимается за точку оси Y.

### 3.8 Формирование системы координат

После нахождения всех необходимых точек формируется локальная система координат.

Пусть:  $O$  - начало координат,  $X$  - точка на оси X,  $Y$  - точка на оси Y.

Тогда единичный вектор оси X определяется выражением

$$\vec{x} = \frac{X - O}{|X - O|}.$$

Аналогично вычисляется единичный вектор оси Y:

$$\vec{y} = \frac{Y - O}{|Y - O|}.$$

Полученная структура содержит:

- координаты начала системы;
- направление оси X;
- направление оси Y.

Таким образом каждая обнаруженная система координат описывается набором

$$O\vec{x}, \vec{y}$$

### 3.9 Визуализация результатов

Для проверки корректности работы алгоритма реализован модуль визуализации. На исходном изображении отображаются:

- обнаруженные узлы сетки;
- специальные маркеры;

- начала координат;
- оси X;
- оси Y;
- номер найденной системы координат.

Ось X отображается красным цветом, ось Y — синим цветом.

Визуальный контроль результатов позволяет оперативно выявлять ошибки восстановления структуры сетки и корректировать параметры алгоритма.

### 3.10 Выводы

В результате был разработан алгоритм восстановления регулярной структуры калибровочной карты и автоматического определения локальных координатных систем.

Предложенный подход сочетает методы коррекции перспективных искажений, пространственного поиска ближайших соседей и анализа топологии сетки. Использование информации о регулярной структуре карты позволяет надёжно восстанавливать положение маркеров даже при наличии ошибок детектирования и частичных пропусков точек.

Разработанный алгоритм является основой для дальнейшего выполнения измерений и калибровки гальванометрического сканатора установки SLM-печати.

### 3.11 Пользовательский интерфейс и конфигурирование системы

Для управления параметрами алгоритма и визуального контроля процесса обработки в системе реализовано два взаимозаменяемых интерфейса: интерактивный консольный интерфейс (*CLI*) и графический оконный интерфейс (*GUI*).

Управление всеми конфигурационными параметрами централизовано в классе `AppConfig` с использованием механизма датаклассов (`@dataclass`). В перечень настраиваемых параметров входят: геометрические параметры сетки (`grid_rows`, `grid_cols`, `marker_step_mm`), настройки детектора (`pca_roi_size`, `pca_corner_ratio`), пороги фильтрации и режимы визуализации.

#### 3.11.1 Консольный интерфейс (CLI)

Консольный интерфейс реализован в классе `ConsoleInterface` и представляет собой текстовое меню, функционирующее в бесконечном цикле обработки ввода оператора. Основные функции класса включают в себя:

- `_print_menu` - вывод структурированного главного меню настроек в терминал;
- Группу методов редактирования параметров (`_edit_grid`, `_edit_detector`, `_edit_coordinate` и др.), обеспечивающих диалоговый ввод новых значений с возможностью сохранения параметров по умолчанию;
- `_show_config` - интерактивное отображение текущего состояния всех полей конфигурации.

Предусмотрена поддержка аргументов командной строки (флаги `-fast`, `-no-ui`), позволяющая запускать систему в полностью автоматическом режиме без вывода текстового меню, что критически важно для интеграции комплекса в автоматизированные конвейеры обработки данных.

### 3.11.2 Графический интерфейс (GUI)

Для интерактивной работы оператора разработан графический интерфейс на базе библиотеки `tkinter`, инкапсулированный в классе `AppGui`. Архитектура графического окна разделена на две функциональные зоны:

1. **Левая панель параметров:** Содержит сгруппированные блоки ввода (`Entry`, `Radiobutton`, `Checkbutton`) для оперативного изменения настроек сетки, РСА-детектора и режимов отображения. При нажатии кнопки «ЗАПУСТИТЬ ОБРАБОТКУ» метод `_save_config` осуществляет валидацию введенных данных и генерирует объект конфигурации.
2. **Правая панель визуализации:** Предназначена для вывода текущего обрабатываемого кадра. Отображение графической информации реализовано через связку `PIL.Image` и `ImageTk.PhotoImage`.

С целью предотвращения зависания главного графического потока (*Main Loop*) при выполнении ресурсоемких математических операций, конвейер обработки `run_pipeline` запускается в изолированном фоновом потоке средствами модуля `threading.Thread`.

Для синхронизации кадров используется механизм событий `threading.Event()` (объект `next_frame_event`). При обработке кадра выполнение фонового потока блокируется методом `wait()`, а кнопка «Следующий кадр» переводится в активное состояние (`tk.NORMAL`). Нажатие оператором кнопки активирует событие (`set()`), возобновляя вычисления для следующего изображения, что позволяет детально анализировать результаты работы алгоритмов на каждом шаге.

## 4 Экспериментальные исследования

### 4.1 Описание программного комплекса

Для верификации разработанных алгоритмов была реализована программная система на языке Python с использованием библиотек компьютерного зрения (OpenCV), научных вычислений (NumPy, SciPy) и графического интерфейса (Tkinter).

Архитектура системы включает в себя следующие ключевые компоненты:

- **Binarizer** - модуль адаптивной предобработки, выполняющий нормализацию яркости, CLAHE-контрастирование, выделение области черной пластины (*Black Plate*) с помощью пороговой обработки Оцу и морфологическую фильтрацию.
- **PcaDetector** - детектор центров перекрестий на основе анализа главных компонент локального тензора градиентов с последующим субпиксельным уточнением (`cornerSubPix`).
- **PerspectiveCorrector** - модуль автоматического восстановления топологии сетки, использующий робастный алгоритм RANSAC для оценивания матрицы гомографии между искаженным изображением и эталонной математической моделью сетки в миллиметрах, а также венгерский алгоритм (`linear_sum_assignment`) для сопоставления точек.
- **CoordinateSystemDetector** - модуль локализации локальных систем координат (СК), идентифицирующий специальные маркеры (перекрестия, обведенные окружностью) и вычисляющий направления базисных векторов осей.

## 4.2 Методика и результаты экспериментов

Экспериментальные исследования проводились на серии изображений калибровочных сеток с размеренностью  $7 \times 7$  элементов и шагом маркеров 35.0 мм. Программа тестировалась в двух режимах: *Grid Mode* (полный конвейер поиска регулярной структуры) и *Single Cross Mode* (изолированный поиск центрального маркера).

В ходе экспериментов было установлено, что модуль предобработки *Binarizer* эффективно изолирует рабочее поле, отсекая контуры, не удовлетворяющие геометрическим критериям: площади (в диапазоне от 0.005% до 1.5% от площади пластины), коэффициенту формы (*aspect ratio* в пределах  $[0.4; 2.5]$ ) и заполнению (*extent*  $< 0.85$ ).

Алгоритм робастной оценки гомографии на базе RANSAC продемонстрировал высокую устойчивость к перспективным искажениям и пропускам точек: даже при частичном перекрытии маркеров система успешно аппроксимировала недостающие узлы сетки (*restored points*) за счет обратного проективного преобразования.

## 4.3 Оценка производительности

Дополнительно была проведена оценка временных характеристик работы алгоритма. Время сквозной обработки одного кадра определялось как интервал между считыванием изображения и финальным построением координатных систем.

В состав измеряемого времени входили следующие этапы:

- цветовая конвертация и программное кадрирование области пластины;
- вычисление Top-Nat морфологии и бинаризация;
- фильтрация связанных компонентов и поиск контуров;
- субпиксельная локализация центров маркеров методом PCA;
- RANSAC-аппроксимация матрицы гомографии и восстановление пропущенных узлов сетки;
- пространственный анализ взаимного расположения маркеров с использованием дерева  $k$ -мерных пространств ( $k$ -d tree) для локализации систем координат.

Проведённые измерения показали, что среднее время обработки одного изображения в режиме *Grid Mode* составляет несколько секунд (в зависимости от разрешения кадра и размеров сетки), что полностью удовлетворяет требованиям систем автоматизированной калибровки, работающих в режиме реального времени.

## 4.4 Выводы по экспериментам

Проведённые экспериментальные исследования подтвердили высокую надежность и инвариантность разработанной программной системы к внешним условиям съемки.

Результаты экспериментов показали, что алгоритм обеспечивает:

1. Устойчивое бинарное выделение маркеров в условиях неравномерного освещения благодаря CLANE и адаптивному размеру морфологических ядер, масштабируемому от диагонали изображения.
2. Корректное восстановление структуры калибровочной сетки при значительных ракурсно-перспективных искажениях.
3. Автоматическое определение начала и ориентации координатных систем на основе селекции окруженных маркеров.

## 5 Заключение

В ходе выполнения работы был спроектирован, программно реализован и экспериментально исследован сквозной алгоритм автоматического распознавания калибровочных регулярных структур. За счет интеграции методов анализа структурного тензора (РСА), субпиксельного уточнения и проективной геометрии (гомографии) удалось достичь высокой точности локализации координатных меток.

Разработанный программный комплекс снабжен как консольным интерфейсом для пакетной автоматизированной обработки, так и интерактивной графической оболочкой (GUI) для оператора. Полученные точностные и временные характеристики подтверждают готовность разработанного математического и программного обеспечения для интеграции в промышленные установки селективного лазерного плавления (SLM-печати) с целью прецизионной калибровки гальванометрических сканаторов.

## 6 Приложение

Весь исходный код системы представлен на [GitHub](#).

```
1 import cv2
2 import numpy as np
3
4
5 class Binarizer:
6
7     def __init__(self):
8         pass
9
10    def _normalize_and_enhance(self, gray_img: np.ndarray) -> np.ndarray:
11        min_val, max_val, _, _ = cv2.minMaxLoc(gray_img)
12        if max_val - min_val > 2:
13            gray_img = cv2.normalize(
14                gray_img,
15                None,
16                alpha=0,
17                beta=255,
18                norm_type=cv2.NORM_MINMAX,
19                dtype=cv2.CV_8U,
20            )
21        clahe = cv2.createCLAHE(clipLimit=8.0, tileGridSize=(8, 8))
22        return clahe.apply(gray_img)
23
24    def _crop_to_black_plate(self, img: np.ndarray) -> tuple:
25        gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
26        blurred = cv2.GaussianBlur(gray, (9, 9), 0)
27        _, thresh_plate = cv2.threshold(
28            blurred, 0, 255, cv2.THRESH_BINARY_INV + cv2.THRESH_OTSU
29        )
30        contours, _ = cv2.findContours(
31            thresh_plate, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
32        )
33        if not contours:
34            return img, 0, 0, img.shape[0] * img.shape[1]
35        largest_contour = max(contours, key=cv2.contourArea)
36        plate_area = cv2.contourArea(largest_contour)
37        if plate_area < (img.shape[0] * img.shape[1] * 0.15):
38            return img, 0, 0, img.shape[0] * img.shape[1]
39        x, y, w, h = cv2.boundingRect(largest_contour)
40        padding = max(2, int(w * 0.01))
```

```

40     x_start = max(0, x + padding)
41     y_start = max(0, y + padding)
42     x_end = min(img.shape[1], x + w - padding)
43     y_end = min(img.shape[0], y + h - padding)
44     return img[y_start:y_end, x_start:x_end], x_start, y_start,
plate_area
45     def process(self, img: np.ndarray) -> np.ndarray:
46         cropped_img, offset_x, offset_y, plate_area = self.
_crop_to_black_plate(img)
47         gray_cropped = cv2.cvtColor(cropped_img, cv2.COLOR_BGR2GRAY)
48         equalized = self._normalize_and_enhance(gray_cropped)
49         img_diagonal = np.sqrt(cropped_img.shape[0] ** 2 + cropped_img.shape
[1] ** 2)
50         tophat_ksize = int(img_diagonal * 0.018)
51         if tophat_ksize % 2 == 0:
52             tophat_ksize += 1
53         tophat_ksize = max(9, tophat_ksize)
54         kernel = cv2.getStructuringElement(cv2.MORPH_RECT, (tophat_ksize,
tophat_ksize))
55         tophat = cv2.morphologyEx(equalized, cv2.MORPH_TOPHAT, kernel)
56         _, binary_cropped = cv2.threshold(
57             tophat, 0, 255, cv2.THRESH_BINARY + cv2.THRESH_OTSU
58         )
59         #Размер очищающего ядра зависит от масштаба: 0.2% от диагонали
60         morph_size = max(3, int(img_diagonal * 0.002) | 1) # гарантируем
нечетность через побитовое ИЛИ
61         morph_kernel = cv2.getStructuringElement(cv2.MORPH_ELLIPSE, (
morph_size, morph_size))
62         binary_cropped = cv2.dilate(binary_cropped, morph_kernel, iterations
=1)
63         binary_cropped = cv2.morphologyEx(binary_cropped, cv2.MORPH_OPEN,
morph_kernel)
64         full_binary_mask = np.zeros(img.shape[:2], dtype=np.uint8)
65         full_binary_mask[
66             offset_y: offset_y + binary_cropped.shape[0],
67             offset_x: offset_x + binary_cropped.shape[1],
68         ] = binary_cropped
69         filtered_mask = np.zeros_like(full_binary_mask)
70         contours, _ = cv2.findContours(
71             full_binary_mask, cv2.RETR_EXTERNAL, cv2.CHAIN_APPROX_SIMPLE
72         )
73         min_area = plate_area * 0.00005
74         max_area = plate_area * 0.015
75         for cnt in contours:
76             area = cv2.contourArea(cnt)
77             if area < min_area or area > max_area:
78                 continue
79             perimeter = cv2.arcLength(cnt, True)
80             if perimeter <= 0:
81                 continue
82             x, y, w, h = cv2.boundingRect(cnt)
83             aspect_ratio = w / float(h)
84             if not (0.4 <= aspect_ratio <= 2.5):
85                 continue
86             extent = area / float(w * h)
87             if extent > 0.85:
88                 continue
89             cv2.drawContours(filtered_mask, [cnt], -1, 255, -1)
90         return filtered_mask

```

Листинг 1: Препроцессинг



```

1 import cv2
2 import numpy as np
3
4
5 class PcaDetector:
6
7     def __init__(self, roi_size: int = 15, min_corner_ratio: float = 0.12):
8         self.roi_size = roi_size
9         self.min_corner_ratio = min_corner_ratio
10
11     def _refine_subpixel(self, gray_img: np.ndarray, point: tuple) -> tuple:
12         corner = np.array([[point]], dtype=np.float32)
13         criteria = (
14             cv2.TERM_CRITERIA_EPS + cv2.TERM_CRITERIA_MAX_ITER,
15             40,
16             0.001,
17         )
18         try:
19             cv2.cornerSubPix(
20                 gray_img,
21                 corner,
22                 (5, 5),
23                 (-1, -1),
24                 criteria,
25             )
26             return tuple(corner[0, 0])
27         except cv2.error:
28             return point
29
30     def _is_cross_like(self, gray_img: np.ndarray, point: tuple) -> bool:
31         px, py = point
32         half = self.roi_size // 2
33         x1 = max(0, int(px) - half)
34         y1 = max(0, int(py) - half)
35         x2 = min(gray_img.shape[1], int(px) + half + 1)
36         y2 = min(gray_img.shape[0], int(py) + half + 1)
37         roi = gray_img[y1:y2, x1:x2]
38         if roi.shape[0] < 5 or roi.shape[1] < 5:
39             return False
40         roi = roi.astype(np.float32)
41         dx = cv2.Sobel(roi, cv2.CV_32F, 1, 0, ksize=3)
42         dy = cv2.Sobel(roi, cv2.CV_32F, 0, 1, ksize=3)
43         H = np.array(
44             [
45                 [np.sum(dx * dx), np.sum(dx * dy)],
46                 [np.sum(dx * dy), np.sum(dy * dy)],
47             ],
48             dtype=np.float32,
49         )
50         eigenvalues = np.linalg.eigvalsh(H)
51         lambda_min = eigenvalues[0]
52         lambda_max = eigenvalues[1]
53         if lambda_max < 1e-6:
54             return False
55         return (lambda_min / lambda_max) >= self.min_corner_ratio
56
57     def detect_points(
58         self,
59         binary_mask: np.ndarray,
60         gray_img: np.ndarray = None,
61     ) -> list:

```

```

61     if gray_img is None:
62         gray_img = binary_mask
63
64     h_img, w_img = binary_mask.shape[:2]
65
66     contours, _ = cv2.findContours(
67         binary_mask,
68         cv2.RETR_EXTERNAL,
69         cv2.CHAIN_APPROX_SIMPLE,
70     )
71     detected_points = []
72     for cnt in contours:
73         x, y, w, h = cv2.boundingRect(cnt)
74         if (
75             x <= 2
76             or y <= 2
77             or x + w >= w_img - 2
78             or y + h >= h_img - 2
79         ):
80             continue
81         M = cv2.moments(cnt)
82         if M["m00"] == 0:
83             continue
84         cx = M["m10"] / M["m00"]
85         cy = M["m01"] / M["m00"]
86         refined_point = self._refine_subpixel(gray_img, (cx, cy))
87         if not self._is_cross_like(gray_img, refined_point):
88             continue
89         detected_points.append(
90             (
91                 float(refined_point[0]),
92                 float(refined_point[1]),
93             )
94         )
95     return detected_points
96
97 def detect_single_cross_center(self, gray_img: np.ndarray):
98     if gray_img is None:
99         return None
100     img = gray_img.astype(np.float32)
101     dx = cv2.Sobel(img, cv2.CV_32F, 1, 0, ksize=3)
102     dy = cv2.Sobel(img, cv2.CV_32F, 0, 1, ksize=3)
103     mag = np.sqrt(dx * dx + dy * dy)
104     if np.max(mag) < 1e-6:
105         return None
106     mask = mag > (0.6 * np.max(mag))
107     if np.sum(mask) < 50:
108         return None
109     ys, xs = np.where(mask)
110     pts = np.column_stack((xs, ys)).astype(np.float32)
111     if len(pts) < 20:
112         return None
113     dx_v = dx[mask]
114     dy_v = dy[mask]
115     vert_mask = np.abs(dx_v) > np.abs(dy_v)
116     horiz_mask = ~vert_mask
117     vert_pts = pts[vert_mask]
118     horiz_pts = pts[horiz_mask]
119     if len(vert_pts) < 10 or len(horiz_pts) < 10:
120         return None

```

```

121     vx, vy, x0, y0 = cv2.fitLine(
122         vert_pts, cv2.DIST_L2, 0, 0.01, 0.01
123     )
124     hx, hy, x1, y1 = cv2.fitLine(
125         horiz_pts, cv2.DIST_L2, 0, 0.01, 0.01
126     )
127     vx, vy, x0, y0 = vx.item(), vy.item(), x0.item(), y0.item()
128     hx, hy, x1, y1 = hx.item(), hy.item(), x1.item(), y1.item()
129     A1, B1 = -vy, vx
130     C1 = -(A1 * x0 + B1 * y0)
131     A2, B2 = -hy, hx
132     C2 = -(A2 * x1 + B2 * y1)
133     det = A1 * B2 - A2 * B1
134     if abs(det) < 1e-6:
135         return None
136     cx = (B1 * C2 - B2 * C1) / det
137     cy = (C1 * A2 - C2 * A1) / det
138     return float(cx), float(cy)

```

Листинг 2: Нахождение центров меток

```

1 import cv2
2 import numpy as np
3 from scipy.spatial import cKDTree
4 from scipy.optimize import linear_sum_assignment
5
6
7 class PerspectiveCorrector:
8     def __init__(
9         self,
10         grid_size=(7, 7),
11         actual_step_mm=35.0,
12         ransac_iterations=500,
13     ):
14         self.grid_size = grid_size
15         self.actual_step_mm = actual_step_mm
16         self.ransac_iterations = ransac_iterations
17         self.ideal_mm_grid = self._generate_grid()
18     def _generate_grid(self):
19         rows, cols = self.grid_size
20         x = np.arange(cols) * self.actual_step_mm
21         y = np.arange(rows) * self.actual_step_mm
22         xx, yy = np.meshgrid(x, y)
23         return np.column_stack(
24             [xx.ravel(), yy.ravel()]
25         ).astype(np.float32)
26     def _build_mapping(
27         self,
28         transformed_points,
29         threshold_mm,
30     ):
31         cost_matrix = np.linalg.norm(
32             self.ideal_mm_grid[:, None, :]
33             - transformed_points[None, :, :],
34             axis=2,
35         )
36         rows, cols = linear_sum_assignment(cost_matrix)
37         mapping = {}
38         for grid_idx, point_idx in zip(rows, cols):
39             if cost_matrix[grid_idx, point_idx] <= threshold_mm:
40                 mapping[grid_idx] = point_idx

```

```

41     return mapping
42     def _estimate_homography_robust(
43         self,
44         points,
45     ):
46         if len(points) < 6:
47             return None, self.ideal_mm_grid, {}
48         threshold_mm = self.actual_step_mm * 0.45
49         best_h = None
50         best_mapping = {}
51         best_inliers = 0
52         rng = np.random.default_rng(42)
53         tree = cKDTree(points)
54         for _ in range(self.ransac_iterations):
55             base_idx = rng.integers(len(points))
56             _, neighbor_ids = tree.query(
57                 points[base_idx],
58                 k=min(12, len(points)),
59             )
60             if len(np.atleast_1d(neighbor_ids)) < 4:
61                 continue
62             sample_ids = rng.choice(
63                 np.atleast_1d(neighbor_ids),
64                 4,
65                 replace=False,
66             )
67             sample_pts = points[sample_ids]
68             sums = sample_pts.sum(axis=1)
69             diffs = np.diff(sample_pts, axis=1).flatten()
70             src_quad = np.array(
71                 [
72                     sample_pts[np.argmin(sums)],
73                     sample_pts[np.argmin(diffs)],
74                     sample_pts[np.argmax(sums)],
75                     sample_pts[np.argmax(diffs)],
76                 ],
77                 dtype=np.float32,
78             )
79             for row in range(self.grid_size[0] - 1):
80                 for col in range(self.grid_size[1] - 1):
81                     dst_quad = np.array(
82                         [
83                             [
84                                 col * self.actual_step_mm,
85                                 row * self.actual_step_mm,
86                             ],
87                             [
88                                 (col + 1) * self.actual_step_mm,
89                                 row * self.actual_step_mm,
90                             ],
91                             [
92                                 (col + 1) * self.actual_step_mm,
93                                 (row + 1) * self.actual_step_mm,
94                             ],
95                             [
96                                 col * self.actual_step_mm,
97                                 (row + 1) * self.actual_step_mm,
98                             ],
99                         ],
100                         dtype=np.float32,

```

```

101         )
102         h = cv2.getPerspectiveTransform(
103             src_quad,
104             dst_quad,
105         )
106         if abs(np.linalg.det(h)) < 1e-8:
107             continue
108         transformed = cv2.perspectiveTransform(
109             np.array([points]),
110             h,
111         )[0]
112         mapping = self._build_mapping(
113             transformed,
114             threshold_mm,
115         )
116         inliers = len(mapping)
117         if inliers > best_inliers:
118             best_inliers = inliers
119             best_h = h
120             best_mapping = mapping
121     if best_h is None or best_inliers < 12:
122         return None, self.ideal_mm_grid, {}
123     src_refine = np.array(
124         [
125             points[idx]
126             for idx in best_mapping.values()
127         ],
128         dtype=np.float32,
129     )
130     dst_refine = np.array(
131         [
132             self.ideal_mm_grid[idx]
133             for idx in best_mapping.keys()
134         ],
135         dtype=np.float32,
136     )
137     refined_h, mask = cv2.findHomography(
138         src_refine,
139         dst_refine,
140         cv2.RANSAC,
141         ransacReprojThreshold=self.actual_step_mm * 0.3,
142     )
143     if refined_h is None:
144         return best_h, self.ideal_mm_grid, best_mapping
145     transformed = cv2.perspectiveTransform(
146         np.array([points]),
147         refined_h,
148     )[0]
149     final_mapping = self._build_mapping(
150         transformed,
151         threshold_mm,
152     )
153     return (
154         refined_h,
155         self.ideal_mm_grid,
156         final_mapping,
157     )
158 def process_and_restore_grid(
159     self,
160     detected_points,

```

```

161 ):
162     if len(detected_points) < 4:
163         return detected_points, []
164     points = np.asarray(
165         detected_points,
166         dtype=np.float32,
167     )
168     (h_matrix, ideal_grid, mapping) = self._estimate_homography_robust(
169         points)
170     if h_matrix is None:
171         return detected_points, []
172     try:
173         h_inv = np.linalg.inv(h_matrix)
174     except np.linalg.LinAlgError:
175         return detected_points, []
176     final_pixel_points = []
177     final_mm_points = []
178     for grid_idx, mm_pt in enumerate(ideal_grid):
179         if grid_idx in mapping:
180             point_idx = mapping[grid_idx]
181             final_pixel_points.append(
182                 tuple(
183                     map(
184                         float,
185                         detected_points[point_idx],
186                     )
187                 )
188             )
189         else:
190             restored = cv2.perspectiveTransform(
191                 np.array([[mm_pt]], dtype=np.float32),
192                 h_inv,
193             )[0][0]
194             final_pixel_points.append(
195                 (
196                     float(restored[0]),
197                     float(restored[1]),
198                 )
199             )
200             final_mm_points.append(mm_pt.tolist())
201     return (final_pixel_points, final_mm_points,)

```

Листинг 3: Работа с геометрией

```

1 import cv2
2 import numpy as np
3
4
5 class Visualizer:
6
7     def __init__(
8         self,
9         marker_color: tuple = (0, 255, 0),
10         radius: int = 5,
11         thickness: int = 2,
12     ):
13         self.marker_color = marker_color
14         self.radius = radius
15         self.thickness = thickness
16         self.dot_color = (0, 0, 255)
17

```

```

18     def draw_detected_points(self, img: np.ndarray, points: list) -> np.
        ndarray:
19         vis_img = img.copy()
20
21         for pt in points:
22             center_x = int(round(pt[0]))
23             center_y = int(round(pt[1]))
24             cv2.circle(
25                 vis_img,
26                 (center_x, center_y),
27                 max(1, int(self.radius * 0.3)),
28                 self.dot_color,
29                 -1
30             )
31
32         return vis_img

```

Листинг 4: Визуализация результатов

```

1 import cv2
2 import numpy as np
3 from scipy.spatial import cKDTree
4
5
6 def detect_circled_points(binary_mask, grid_points):
7     if len(grid_points) == 0:
8         return []
9     grid_points = np.array(grid_points, dtype=np.float32)
10    gray = binary_mask.copy()
11
12    contours, _ = cv2.findContours(
13        gray,
14        cv2.RETR_EXTERNAL,
15        cv2.CHAIN_APPROX_SIMPLE,
16    )
17    debug_vis = cv2.cvtColor(binary_mask, cv2.COLOR_GRAY2BGR)
18    grid_tree = cKDTree(grid_points)
19    matched_grid_centers = {}
20    for cnt in contours:
21        area = cv2.contourArea(cnt)
22        if area < 400:
23            continue
24        M = cv2.moments(cnt)
25        if M["m00"] == 0:
26            continue
27        cx = M["m10"] / M["m00"]
28        cy = M["m01"] / M["m00"]
29        dist, idx = grid_tree.query([cx, cy])
30        if dist < 30.0:
31            grid_pt = tuple(grid_points[idx])
32            matched_grid_centers[grid_pt] = area
33            cv2.drawContours(debug_vis, [cnt], -1, (0, 255, 0), 2)
34            cv2.circle(debug_vis, (int(cx), int(cy)), 3, (0, 0, 255), -1)
35        else:
36            cv2.drawContours(debug_vis, [cnt], -1, (255, 0, 255), 2)
37    cv2.imshow("Detected Circles Debug", debug_vis)
38    cv2.waitKey(1)
39    unique_centers = list(matched_grid_centers.keys())
40    print(f"DEBUG: Итоговых уникальных маркеров для СК: {len(unique_centers)}")
41    return unique_centers

```

Листинг 5: Нахождение маркеров для координатной системы

```

1 import os
2 import json
3 import time
4 import argparse
5 from dataclasses import dataclass, asdict
6 from typing import List, Optional, Tuple
7
8
9 @dataclass
10 class AppConfig:
11     grid_rows: int = 7
12     grid_cols: int = 7
13     marker_step_mm: float = 35.0
14     pca_roi_size: int = 15
15     pca_corner_ratio: float = 0.12
16     cluster_dist: float = 80.0
17     angle_thresh: float = 25.0
18     show_images: bool = True
19     show_statistics: bool = True
20     show_coordinate_systems: bool = True
21     verbose: bool = False
22     start_image: int = 1
23     end_image: int = 10
24     cross_mode: str = "grid"
25     save_images: bool = False
26     save_coords: bool = False
27     output_dir: str = "output"
28     debug: bool = False
29
30
31 class ConsoleInterface:
32
33     def __init__(self):
34         self.config = AppConfig()
35         self.running = True
36
37     def run(self):
38         print("\n" + "=" * 70)
39         print("  CROSS MARKER DETECTOR - Консольный интерфейс")
40         print("=" * 70)
41
42         while self.running:
43             self._print_main_menu()
44             choice = input("\Выберите пункт: ").strip()
45
46             if choice == "1":
47                 self._run_pipeline()
48             elif choice == "2":
49                 self._edit_config()
50             elif choice == "3":
51                 self._load_config_from_json()
52             elif choice == "4":
53                 self._save_config_to_json()
54             elif choice == "5":
55                 self._show_config()
56             elif choice == "6":
57                 self._batch_process()
58             elif choice == "7":
59                 self._generate_example_config()
60             elif choice == "0":

```



```

61         self.running = False
62         print("\Выход из программы...")
63     else:
64         printНеверный(" выбор. Попробуйте снова.")
65
66     def _print_main_menu(self):
67         print("\n" + "-" * 70)
68         print("    ГЛАВНОЕ МЕНЮ")
69         print("-" * 70)
70         print("    1 - Запустить обработку")
71         print("    2 - Редактировать конфигурацию")
72         print("    3 - Загрузить конфигурацию из JSON")
73         print("    4 - Сохранить конфигурацию в JSON")
74         print("    5 - Показать текущую конфигурацию")
75         print("    6 - Изменить режим работы")
76         print("    7 - Сгенерировать пример конфигурации")
77         print("    0 - Выход")
78         print("-" * 70)
79
80     def _print_edit_menu(self):
81         print("\n" + "-" * 70)
82         print("    РЕДАКТИРОВАНИЕ КОНФИГУРАЦИИ")
83         print("-" * 70)
84         print("    1 - Настройки сетки")
85         print("    2 - Настройки детектора")
86         print("    3 - Настройки координатных систем")
87         print("    4 - Настройки отображения")
88         print("    5 - Диапазон изображений")
89         print("    6 - Режим работы")
90         print("    7 - Настройки сохранения")
91         print("    0 - Вернуться в главное меню")
92         print("-" * 70)
93
94     def _edit_config(self):
95         while True:
96             self._print_edit_menu()
97             choice = input("\Выберите пункт: ").strip()
98
99             if choice == "0":
100                 break
101             elif choice == "1":
102                 self._edit_grid_settings()
103             elif choice == "2":
104                 self._edit_detector_settings()
105             elif choice == "3":
106                 self._edit_coordinate_settings()
107             elif choice == "4":
108                 self._edit_display_settings()
109             elif choice == "5":
110                 self._edit_image_range()
111             elif choice == "6":
112                 self._edit_work_mode()
113             elif choice == "7":
114                 self._edit_save_settings()
115             else:
116                 printНеверный(" выбор.")
117
118     def _edit_grid_settings(self):
119         cfg = self.config
120         print("\Настройка сетки")

```

```

121     print("-" * 40)
122
123     v = input(f"    Количество строк [{cfg.grid_rows}]: ").strip()
124     if v:
125         cfg.grid_rows = max(1, int(v))
126
127     v = input(f"    Количество столбцов [{cfg.grid_cols}]: ").strip()
128     if v:
129         cfg.grid_cols = max(1, int(v))
130
131     v = input(f"    Шаг маркеров мм() [{cfg.marker_step_mm}]: ").strip()
132     if v:
133         cfg.marker_step_mm = max(0.1, float(v))
134
135     printНастройки(" сетки обновлены")
136
137     def _edit_detector_settings(self):
138         cfg = self.config
139         print("\Настройки детектора")
140         print("-" * 40)
141
142         v = input(f"    Размер ROI для PCA [{cfg.pca_roi_size}]: ").strip()
143         if v:
144             cfg.pca_roi_size = max(3, int(v))
145
146         v = input(f"    Коэффициент углов [{cfg.pca_corner_ratio}]: ").strip()
147         if v:
148             cfg.pca_corner_ratio = max(0.01, min(1.0, float(v)))
149
150         printНастройки(" детектора обновлены")
151
152     def _edit_coordinate_settings(self):
153         cfg = self.config
154         print("\Настройки координатных систем")
155         print("-" * 40)
156
157         v = input(f"    Расстояние кластеризации [{cfg.cluster_dist}]: ").strip()
158         if v:
159             cfg.cluster_dist = max(1.0, float(v))
160
161         v = input(f"    Порог угла градусы() [{cfg.angle_thresh}]: ").strip()
162         if v:
163             cfg.angle_thresh = max(0.1, min(90.0, float(v)))
164
165         printНастройки(" координатных систем обновлены")
166
167     def _edit_display_settings(self):
168         cfg = self.config
169         print("\Настройки отображения")
170         print("-" * 40)
171
172     (
173         if v:
174             cfg.show_images = bool(int(v))
175
176         v = input(f"    Показывать статистику [{int(cfg.show_statistics)}]: ").
strip()
177         if v:
178             cfg.show_statistics = bool(int(v))

```

```

179
180         v = input(f" Показывать координатные системы [{int(cfg.
show_coordinate_systems)}]: ").strip()
181         if v:
182             cfg.show_coordinate_systems = bool(int(v))
183
184         v = input(f" Подробный вывод [{int(cfg.verbose)}]: ").strip()
185         if v:
186             cfg.verbose = bool(int(v))
187
188         v = input(f" Режим отладки [{int(cfg.debug)}]: ").strip()
189         if v:
190             cfg.debug = bool(int(v))
191
192         printНастройки(" отображения обновлены")
193
194     def _edit_image_range(self):
195         cfg = self.config
196         print("\Диапазон изображений")
197         print("-" * 40)
198
199         v = input(f" Начальный номер [{cfg.start_image}]: ").strip()
200         if v:
201             cfg.start_image = max(0, int(v))
202
203         v = input(f" Конечный номер [{cfg.end_image}]: ").strip()
204         if v:
205             cfg.end_image = max(cfg.start_image, int(v))
206
207         print(f"Диапазон" обновлен: {cfg.start_image} - {cfg.end_image}")
208
209     def _edit_work_mode(self):
210         cfg = self.config
211         print("\Режим работы")
212         print("-" * 40)
213         print(" grid - Обработка сетки маркеров")
214         print(" single - Поиск одиночного крестика")
215         print("-" * 40)
216
217         v = input(f" Режим [{cfg.cross_mode}]: ").strip()
218         if v and v.lower() in ["grid", "single"]:
219             cfg.cross_mode = v.lower()
220             print(f"Режим" обновлен: {cfg.cross_mode})
221         elif v:
222             printНеверный(" режим. Доступны: grid, single")
223
224     def _edit_save_settings(self):
225         cfg = self.config
226         print("\Настройки сохранения")
227         print("-" * 40)
228
229         v = input(f" Сохранять изображения [{int(cfg.save_images)}]: ").strip()
230         if v:
231             cfg.save_images = bool(int(v))
232
233         v = input(f" Сохранять координаты [{int(cfg.save_coords)}]: ").strip()
234         if v:
235             cfg.save_coords = bool(int(v))
236
237         v = input(f" Папка для результатов [{cfg.output_dir}]: ").strip()

```

```

238         if v:
239             cfg.output_dir = v
240
241         printНастройки(" сохранения обновлены")
242
243     def _save_config_to_json(self):
244         print("\Сохранение конфигурации")
245         print("-" * 40)
246
247         default_name = "config.json"
248         filename = input(f" Имя файла [{default_name}]: ").strip()
249         if not filename:
250             filename = default_name
251
252         try:
253             with open(filename, 'w', encoding='utf-8') as f:
254                 json.dump(asdict(self.config), f, indent=4, ensure_ascii=
False)
255                 print(fКонфигурация" сохранена в {filename}")
256         except Exception as e:
257             print(fОшибка" сохранения: {e}")
258
259     def _load_config_from_json(self):
260         print("\Загрузка конфигурации")
261         print("-" * 40)
262
263         default_name = "config.json"
264         filename = input(f" Имя файла [{default_name}]: ").strip()
265         if not filename:
266             filename = default_name
267
268         if not os.path.exists(filename):
269             print(fФайл" {filename} не найден")
270             return
271
272         try:
273             with open(filename, 'r', encoding='utf-8') as f:
274                 config_data = json.load(f)
275
276                 cfg = self.config
277                 for key, value in config_data.items():
278                     if hasattr(cfg, key):
279                         setattr(cfg, key, value)
280
281                 print(fКонфигурация" загружена из {filename}")
282                 self._show_config()
283         except Exception as e:
284             print(fОшибка" загрузки: {e}")
285
286     def _show_config(self):
287         print("\n" + "=" * 70)
288         print(" ТЕКУЩАЯ КОНФИГУРАЦИЯ")
289         print("=" * 70)
290
291         cfg = self.config
292         print("\НАСТРОЙКИ СЕТКИ:")
293         print(f" Строки: {cfg.grid_rows}")
294         print(f" Столбцы: {cfg.grid_cols}")
295         print(f" Шаг мм(): {cfg.marker_step_mm}")
296

```

```

297     print("\НАСТРОЙКИ ДЕТЕКТОРА:")
298     print(f"    Размер ROI:          {cfg.pca_roi_size}")
299     print(f"    Коэф. углов:          {cfg.pca_corner_ratio}")
300
301     print("\НАСТРОЙКИ КООРДИНАТНЫХ СИСТЕМ:")
302     print(f"    Кластеризация:        {cfg.cluster_dist}")
303     print(f"    Порог угла:          {cfg.angle_thresh} град")
304
305     print("\НАСТРОЙКИ ОТОБРАЖЕНИЯ:")
306     print(f"    Показ. картинок:      {cfg.show_images}")
307     print(f"    Статистика:           {cfg.show_statistics}")
308     print(f"    Коорд. системы:       {cfg.show_coordinate_systems}")
309     print(f"    Подробный вывод:      {cfg.verbose}")
310     print(f"    Отладка:              {cfg.debug}")
311
312     print("\НАСТРОЙКИ ИЗОБРАЖЕНИЙ:")
313     print(f"    Диапазон:             {cfg.start_image} - {cfg.end_image}")
314     print(f"    Режим:                {cfg.cross_mode}")
315
316     print("\НАСТРОЙКИ СОХРАНЕНИЯ:")
317     print(f"    Сохранять фото:       {cfg.save_images}")
318     print(f"    Сохранять коорд:      {cfg.save_coords}")
319     print(f"    Папка результатов:    {cfg.output_dir}")
320
321     print("\n" + "=" * 70)
322
323 def _generate_example_config(self):
324     print("\Генерация примера конфигурации")
325     print("-" * 40)
326
327     filename = input("    Имя файла [example_config.json]: ").strip()
328     if not filename:
329         filename = "example_config.json"
330
331     example_config = {
332         "grid_rows": 7,
333         "grid_cols": 7,
334         "marker_step_mm": 35.0,
335         "pca_roi_size": 15,
336         "pca_corner_ratio": 0.12,
337         "cluster_dist": 80.0,
338         "angle_thresh": 25.0,
339         "show_images": True,
340         "show_statistics": True,
341         "show_coordinate_systems": True,
342         "verbose": False,
343         "debug": False,
344         "start_image": 1,
345         "end_image": 10,
346         "cross_mode": "grid",
347         "save_images": False,
348         "save_coords": False,
349         "output_dir": "output"
350     }
351
352     try:
353         with open(filename, 'w', encoding='utf-8') as f:
354             json.dump(example_config, f, indent=4, ensure_ascii=False)
355         print(fПример" конфигурации сохранен в {filename}")
356         print("\Содержимое примера:")

```

```

357         print(json.dumps(example_config, indent=2, ensure_ascii=False))
358     except Exception as e:
359         print(f"Ошибка" сохранения: {e}")
360
361     def _run_pipeline(self):
362         print("\n" + "=" * 70)
363         print("    ЗАПУСК ОБРАБОТКИ")
364         print("=" * 70)
365
366         self._show_config()
367
368         confirm = input("\nЗапустить обработку с этими настройками? (y/n): ").
strip().lower()
369         if confirm != 'y':
370             print(Запуск(" отменен"))
371             return
372
373         print("\nЗапуск обработки...")
374         start_time = time.time()
375
376         try:
377             from main import run_pipeline
378
379             run_pipeline(
380                 self.config,
381                 save_images=self.config.save_images,
382                 output_dir=self.config.output_dir,
383                 save_coords=self.config.save_coords
384             )
385
386             elapsed = time.time() - start_time
387             print(f"\nОбработка завершена за {elapsed:.2f} секунд")
388
389         except ImportError as e:
390             print(f"Ошибка" импорта: {e})
391             print("    Убедитесь, что файл main.py существует и содержит функцию
run_pipeline")
392         except Exception as e:
393             print(f"Ошибка" обработки: {e})
394             if self.config.debug:
395                 import traceback
396                 traceback.print_exc()
397
398     def _batch_process(self):
399         print("\nПАКЕТНАЯ ОБРАБОТКА")
400         print("=" * 70)
401
402         folder = input(Путь(" к папке с изображениями [data/]: ").strip())
403         if not folder:
404             folder = "data/"
405
406         if not os.path.exists(folder):
407             print(f"Папка" {folder} не найдена")
408             return
409
410         valid_extensions = ('.jpg', '.jpeg', '.png', '.bmp', '.tiff')
411         images = sorted([
412             f for f in os.listdir(folder)
413             if f.lower().endswith(valid_extensions)
414         ])

```

```

415
416     if not images:
417         print(fB" папке {folder} не найдено изображений")
418         return
419
420     print(f"\Найдено изображений: {len(images)}")
421     for i, img in enumerate(images, 1):
422         print(f"  {i}. {img}")
423
424     confirm = input("\Обработать все изображения? (y/n): ").strip().lower()
425     if confirm != 'y':
426         printОтменено("")
427         return
428
429     self.config.start_image = 1
430     self.config.end_image = len(images)
431
432     print(f"\Запуск пакетной обработки ({len(images)} изображений)...")
433     start_time = time.time()
434
435     try:
436         from main import run_pipeline
437
438         run_pipeline(
439             self.config,
440             save_images=self.config.save_images,
441             output_dir=self.config.output_dir,
442             save_coords=self.config.save_coords
443         )
444
445         elapsed = time.time() - start_time
446         print(f"\Пакетная обработка завершена за {elapsed:.2f} секунд")
447
448     except Exception as e:
449         print(fОшибка" пакетной обработки: {e}")
450         if self.config.debug:
451             import traceback
452             traceback.print_exc()
453
454
455 def main():
456     parser = argparse.ArgumentParser(
457         description="Cross Marker Detector - Консольный интерфейс",
458         formatter_class=argparse.RawDescriptionHelpFormatter,
459         epilog="""Примеры
460 использования:
461 python console.py
462 python console.py --config config.json
463 python console.py --mode grid --start 1 --end 10 --save-images
464 """)
465
466
467     parser.add_argument(
468         '--config',
469         type=str,
470         helpПуть= ' к JSON файлу конфигурации'
471     )
472     parser.add_argument(
473         '--mode',
474         type=str,

```

```

475     choices=['grid', 'single'],
476     helpРежим=' работы'
477 )
478 parser.add_argument(
479     '--start',
480     type=int,
481     helpНачальный=' номер изображения'
482 )
483 parser.add_argument(
484     '--end',
485     type=int,
486     helpКонечный=' номер изображения'
487 )
488 parser.add_argument(
489     '--save-images',
490     action='store_true',
491     helpСохранять=' обработанные изображения'
492 )
493 parser.add_argument(
494     '--save-coords',
495     action='store_true',
496     helpСохранять=' координаты точек'
497 )
498 parser.add_argument(
499     '--output',
500     type=str,
501     default='output',
502     helpПапка=' для сохранения результатов'
503 )
504
505 args = parser.parse_args()
506
507 ui = ConsoleInterface()
508
509 if args.config:
510     try:
511         with open(args.config, 'r', encoding='utf-8') as f:
512             config_data = json.load(f)
513             for key, value in config_data.items():
514                 if hasattr(ui.config, key):
515                     setattr(ui.config, key, value)
516             print(f"Конфигурация" загружена из {args.config}")
517     except Exception as e:
518         print(f"Ошибка" загрузки конфигурации: {e}")
519
520 if args.mode:
521     ui.config.cross_mode = args.mode
522
523 if args.start:
524     ui.config.start_image = args.start
525
526 if args.end:
527     ui.config.end_image = args.end
528
529 if args.save_images:
530     ui.config.save_images = True
531
532 if args.save_coords:
533     ui.config.save_coords = True
534

```



```

535     if args.output:
536         ui.config.output_dir = args.output
537
538     ui.run()
539
540
541 if __name__ == "__main__":
542     main()

```

Листинг 6: Консольный интерфейс

```

1 import numpy as np
2 from scipy.spatial import cKDTree
3
4
5 class CoordinateSystemDetector:
6     def __init__(self, angle_thresh=25.0):
7         self.angle_thresh = angle_thresh
8     def _build_frame(self, O, X, Y):
9         x_axis = X - O
10        x_axis = x_axis / (np.linalg.norm(x_axis) + 1e-9)
11        y_axis = Y - O
12        y_axis = y_axis / (np.linalg.norm(y_axis) + 1e-9)
13        return {
14            "origin": O,
15            "x_axis": x_axis,
16            "y_axis": y_axis,
17        }
18    def detect(self, detected_points, circled_points, grid_rows=7, grid_cols
19              =7):
20        if len(detected_points) == 0 or len(circled_points) == 0:
21            return []
22        detected_points = np.array(detected_points, dtype=np.float32)
23        circled_points = np.array(circled_points, dtype=np.float32)
24        grid_tree = cKDTree(detected_points)
25        marked_indices = []
26        for cp in circled_points:
27            dist, idx = grid_tree.query(cp)
28            if dist < 30.0:
29                row = idx // grid_cols
30                col = idx % grid_cols
31                marked_indices.append((row, col, idx))
32        frames = []
33        for o_row, o_col, o_idx in marked_indices:
34            O = detected_points[o_idx]
35            x_idx = None
36            y_idx = None
37            for r, c, idx in marked_indices:
38                if idx == o_idx:
39                    continue
40                if abs(r - o_row) + abs(c - o_col) == 1:
41                    x_idx = idx
42                    x_dir = (r - o_row, c - o_col)
43                    break
44            if x_idx is None:
45                continue
46            y_dir_target = (-x_dir[1], x_dir[0])
47            y_dir_target_inv = (x_dir[1], -x_dir[0])
48            for r, c, idx in marked_indices:
49                if idx == o_idx or idx == x_idx:
50                    continue

```

```

50         current_dir = (r - o_row, c - o_col)
51         if current_dir == (y_dir_target[0] * 2, y_dir_target[1] * 2)
    or \
52             current_dir == (y_dir_target_inv[0] * 2,
y_dir_target_inv[1] * 2):
53             y_idx = idx
54             break
55         if y_idx is None:
56             continue
57         X = detected_points[x_idx]
58         Y = detected_points[y_idx]
59         frames.append(self._build_frame(0, X, Y))
60     return frames

```

Листинг 7: Нахождение координатной системы

```

1 import tkinter as tk
2 from tkinter import ttk, messagebox
3 from PIL import Image, ImageTk
4 import cv2
5 import threading
6 from console import AppConfig
7
8
9 class AppGui:
10     def __init__(self, run_pipeline_callback):
11         self.config = AppConfig()
12         self.run_pipeline_callback = run_pipeline_callback
13         self.root = tk.Tk()
14         self.root.title("Настройки проекта и обработка")
15         self.root.geometry("1200x800")
16         self._init_variables()
17         self._create_widgets()
18
19     def _init_variables(self):
20         self.grid_rows = tk.IntVar(value=self.config.grid_rows)
21         self.grid_cols = tk.IntVar(value=self.config.grid_cols)
22         self.marker_step_mm = tk.DoubleVar(value=self.config.marker_step_mm)
23         self.pca_roi_size = tk.IntVar(value=self.config.pca_roi_size)
24         self.pca_corner_ratio = tk.DoubleVar(value=self.config.
pca_corner_ratio)
25         self.cluster_dist = tk.DoubleVar(value=self.config.cluster_dist)
26         self.angle_thresh = tk.DoubleVar(value=self.config.angle_thresh)
27         self.show_images = tk.BooleanVar(value=self.config.show_images)
28         self.show_statistics = tk.BooleanVar(value=self.config.
show_statistics)
29         self.show_coordinate_systems = tk.BooleanVar(value=self.config.
show_coordinate_systems)
30         self.verbose = tk.BooleanVar(value=self.config.verbose)
31         self.start_image = tk.IntVar(value=self.config.start_image)
32         self.end_image = tk.IntVar(value=self.config.end_image)
33         self.cross_mode = tk.StringVar(value=self.config.cross_mode)
34
35     def _create_widgets(self):
36         main_frame = ttk.Frame(self.root, padding=10)
37         main_frame.pack(fill=tk.BOTH, expand=True)
38         left_panel = ttk.LabelFrame(main_frame, text="Параметры конфигурации ",
padding=10)
39         left_panel.pack(side=tk.LEFT, fill=tk.Y, padx=5, pady=5)
40         grid_frame = ttk.LabelFrame(left_panel, text="Размеры сетки", padding
=5)

```

```

41     grid_frame.pack(fill=tk.X, pady=5)
42     self._add_row(grid_frame, "Rows:", self.grid_rows)
43     self._add_row(grid_frame, "Cols:", self.grid_cols)
44     self._add_row(grid_frame, "Step (mm):", self.marker_step_mm)
45     det_frame = ttk.LabelFrame(left_panel, textНастройки=" детектора",
padding=5)
46     det_frame.pack(fill=tk.X, pady=5)
47     self._add_row(det_frame, "PCA ROI size:", self.pca_roi_size)
48     self._add_row(det_frame, "PCA Corner ratio:", self.pca_corner_ratio)
49     coord_frame = ttk.LabelFrame(left_panel, textКоординатные=" системы",
padding=5)
50     coord_frame.pack(fill=tk.X, pady=5)
51     self._add_row(coord_frame, "Cluster dist:", self.cluster_dist)
52     self._add_row(coord_frame, "Angle thresh:", self.angle_thresh)
53     work_frame = ttk.LabelFrame(left_panel, textРежим=" работы и кадры",
padding=5)
54     work_frame.pack(fill=tk.X, pady=5)
55     self._add_row(work_frame, "Start image:", self.start_image)
56     self._add_row(work_frame, "End image:", self.end_image)
57     ttk.Label(work_frame, text="Cross mode:").pack(anchor=tk.W)
58     ttk.Radiobutton(work_frame, text="grid", value="grid", variable=self
.cross_mode).pack(anchor=tk.W, padx=20)
59     ttk.Radiobutton(work_frame, text="single", value="single", variable=
self.cross_mode).pack(anchor=tk.W, padx=20)
60     view_frame = ttk.LabelFrame(left_panel, textОтображение="", padding=5)
61     view_frame.pack(fill=tk.X, pady=5)
62     ttk.Checkbutton(view_frame, textПоказывать=" изображения", variable=self
.show_images).pack(anchor=tk.W)
63     ttk.Checkbutton(view_frame, textПоказывать=" статистику", variable=self.
show_statistics).pack(anchor=tk.W)
64     ttk.Checkbutton(view_frame, textКоординатные=" системы", variable=self.
show_coordinate_systems).pack(
65         anchor=tk.W)
66     ttk.Checkbutton(view_frame, text="Verbose Логи()", variable=self.
verbose).pack(anchor=tk.W)
67     btn_start = ttk.Button(left_panel, textЗАПУСТИТЬ=" ОБРАБОТКУ", command=
self._on_start_click)
68     btn_start.pack(fill=tk.X, pady=15, ipady=5)
69     right_panel = ttk.Frame(main_frame)
70     right_panel.pack(side=tk.RIGHT, fill=tk.BOTH, expand=True, padx=5,
pady=5)
71     self.image_label = ttk.Label(right_panel, text="42",
72         anchor=tk.CENTER, relief="solid")
73     self.image_label.pack(fill=tk.BOTH, expand=True, pady=5)
74     self.btn_next = ttk.Button(right_panel, textСледующий=" кадр", state=
tk.DISABLED, command=self._on_next_click)
75     self.btn_next.pack(fill=tk.X, pady=2)
76     self.next_frame_event = threading.Event()
77
78     def _add_row(self, frame, label_text, text_var):
79         row = ttk.Frame(frame)
80         row.pack(fill=tk.X, pady=2)
81         ttk.Label(row, text=label_text, width=15, anchor=tk.W).pack(side=tk.
LEFT)
82         ttk.Entry(row, textvariable=text_var, width=10).pack(side=tk.RIGHT,
expand=True, fill=tk.X)
83
84     def _save_config(self):
85         try:
86             self.config.grid_rows = int(self.grid_rows.get())

```

```

87         self.config.grid_cols = int(self.grid_cols.get())
88         self.config.marker_step_mm = float(self.marker_step_mm.get())
89         self.config.pca_roi_size = int(self.pca_roi_size.get())
90         self.config.pca_corner_ratio = float(self.pca_corner_ratio.get())
91     )
92     self.config.cluster_dist = float(self.cluster_dist.get())
93     self.config.angle_thresh = float(self.angle_thresh.get())
94     self.config.show_images = bool(self.show_images.get())
95     self.config.show_statistics = bool(self.show_statistics.get())
96     self.config.show_coordinate_systems = bool(self.
show_coordinate_systems.get())
97     self.config.verbose = bool(self.verbose.get())
98     self.config.start_image = int(self.start_image.get())
99     self.config.end_image = int(self.end_image.get())
100     self.config.cross_mode = str(self.cross_mode.get())
101 except ValueError:
102     messagebox.showerror("Ошибка", "Проверьте" правильность ввода числовых
параметров!")
103     return False
104     return True
105
106 def _on_start_click(self):
107     if not self._save_config():
108         return
109     self.next_frame_event.clear()
110     threading.Thread(target=self.run_pipeline_callback, args=(self.
config, self.update_image_ui),
111                     daemon=True).start()
112
113 def update_image_ui(self, img_bgr, title=""):
114     if img_bgr is None:
115         return
116     img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
117     img_pil = Image.fromarray(img_rgb)
118     img_pil.thumbnail((700, 500))
119     img_tk = ImageTk.PhotoImage(image=img_pil)
120     self.root.after(0, self._set_image, img_tk)
121     if self.config.show_images:
122         self.root.after(0, lambda: self.btn_next.config(state=tk.NORMAL)
)
123
124     self.next_frame_event.wait()
125     self.next_frame_event.clear()
126     self.root.after(0, lambda: self.btn_next.config(state=tk.
DISABLED))
127
128 def _set_image(self, img_tk):
129     self.image_label.config(image=img_tk, text="")
130     self.image_label.image = img_tk
131
132 def _on_next_click(self):
133     self.next_frame_event.set()
134
135 def mainloop(self):
136     self.root.mainloop()

```

Листинг 8: Графический интерфейс

```

1 import time
2 import cv2
3 import argparse
4 import json
5 import os
6 import numpy as np
7 from console import ConsoleInterface

```

```

8 from gui import AppGui
9 from preprocess.preprocess import Binarizer
10 from detection.PCA import PcaDetector
11 from geometry import PerspectiveCorrector
12 from utils.visual import Visualizer
13 from detection.detect_circles import detect_circled_points
14 from origins import CoordinateSystemDetector
15 from dataclasses import dataclass
16
17
18 @dataclass
19 class AppConfig:
20     grid_rows: int = 7
21     grid_cols: int = 7
22     marker_step_mm: float = 35.0
23     pca_roi_size: int = 15
24     pca_corner_ratio: float = 0.12
25     cluster_dist: float = 80.0
26     angle_thresh: float = 25.0
27     show_images: bool = True
28     show_statistics: bool = True
29     show_coordinate_systems: bool = True
30     verbose: bool = False
31     start_image: int = 1
32     end_image: int = 10
33     cross_mode: str = "grid"
34     save_images: bool = False
35     save_coords: bool = False
36     output_dir: str = "output"
37     debug: bool = False
38
39
40 def parse_args():
41     p = argparse.ArgumentParser()
42     p.add_argument("--fast", action="store_true")
43     p.add_argument("--no-ui", action="store_true")
44     p.add_argument("--gui", action="store_true", helpЗапустить=" оконный GUI интерфейс")
45     p.add_argument("--config", type=str, helpПуть=" к JSON файлу с настройками")
46     p.add_argument("--output-dir", type=str, default="output", helpПапка=" для сохранения результатов")
47     p.add_argument("--save-images", action="store_true", helpСохранять=" обработанные изображения")
48     p.add_argument("--save-coords", action="store_true", helpСохранять=" координаты точек в CSV")
49     return p.parse_args()
50
51
52 def load_config_from_json(json_path):
53     if not os.path.exists(json_path):
54         print(fФайл {json_path} не найден, используются стандартные настройки")
55         return AppConfig()
56
57     try:
58         with open(json_path, 'r', encoding='utf-8') as f:
59             config_data = json.load(f)
60
61         cfg = AppConfig()
62         for key, value in config_data.items():
63             if hasattr(cfg, key):

```

```

64         setattr(cfg, key, value)
65
66         print(f"Конфигурация" загружена из {json_path}")
67         return cfg
68     except Exception as e:
69         print(f"Ошибка" загрузки конфигурации: {e}")
70         return AppConfig()
71
72
73 def save_config_to_json(cfg, json_path="config.json"):
74     try:
75         with open(json_path, 'w', encoding='utf-8') as f:
76             json.dump(vars(cfg), f, indent=4, ensure_ascii=False)
77             print(f"Конфигурация" сохранена в {json_path}")
78     except Exception as e:
79         print(f"Ошибка" сохранения конфигурации: {e}")
80
81
82 def save_coordinates_to_csv(coordinate_systems, pixel_points, filename,
83                             image_idx):
84     import csv
85
86     os.makedirs(os.path.dirname(filename), exist_ok=True)
87
88     with open(filename, 'w', newline='', encoding='utf-8') as csvfile:
89         writer = csv.writer(csvfile)
90         writer.writerow(['Image', 'Point_Type', 'ID', 'X', 'Y'])
91
92         # Проверяем тип pixel_points и преобразуем в список если нужно
93         if isinstance(pixel_points, np.ndarray):
94             points_list = pixel_points.tolist()
95         else:
96             points_list = list(pixel_points)
97
98         for idx, point in enumerate(points_list):
99             writer.writerow([image_idx, 'grid_point', idx, point[0], point
100 [1]])
101
102         for idx, system in enumerate(coordinate_systems):
103             writer.writerow([image_idx, 'origin', idx, system['origin'][0],
104 system['origin'][1]])
105             writer.writerow([image_idx, 'x_axis', idx, system['x_axis'][0],
106 system['x_axis'][1]])
107             writer.writerow([image_idx, 'y_axis', idx, system['y_axis'][0],
108 system['y_axis'][1]])
109
110
111 def run_pipeline(cfg, ui_image_callback=None, save_images=False, output_dir
112 = "output", save_coords=False):
113     binarizer = Binarizer()
114
115     detector = PcaDetector(
116         roi_size=cfg.pca_roi_size,
117         min_corner_ratio=cfg.pca_corner_ratio,
118     )
119
120     coord_detector = CoordinateSystemDetector(
121         angle_thresh=cfg.angle_thresh,
122     )

```

```

118     visualizer = Visualizer(
119         marker_color=(0, 255, 0),
120         radius=1,
121         thickness=2,
122     )
123
124     if save_images or save_coords:
125         os.makedirs(output_dir, exist_ok=True)
126         if save_images:
127             images_dir = os.path.join(output_dir, "images")
128             os.makedirs(images_dir, exist_ok=True)
129         if save_coords:
130             coords_dir = os.path.join(output_dir, "coordinates")
131             os.makedirs(coords_dir, exist_ok=True)
132
133     all_results = []
134
135     for i in range(cfg.start_image, cfg.end_image + 1):
136         image_path = f"data/photo_{i}.jpg"
137         img = cv2.imread(image_path)
138
139         if img is None:
140             print(f"Не удалось открыть {image_path}")
141             continue
142
143         if cfg.verbose:
144             print(f"\066работка кадра: {image_path}")
145             start_time = time.time()
146             gray = cv2.cvtColor(img, cv2.COLOR_BGR2GRAY)
147
148         if cfg.cross_mode == "single":
149             center = detector.detect_single_cross_center(gray)
150             dt = time.time() - start_time
151             if cfg.show_statistics:
152                 print("-" * 50)
153                 print("MODE: SINGLE")
154                 print("center:", center)
155                 print(f"time: {dt:.4f} sec")
156                 print("-" * 50)
157             if center is not None:
158                 vis = img.copy()
159                 cv2.circle(vis, (int(center[0]), int(center[1])), 5, (0, 0,
255), -1)
160
161                 if save_images:
162                     save_path = os.path.join(images_dir, f"single_{i}.jpg")
163                     cv2.imwrite(save_path, vis)
164                     print(f"Изображение" сохранено: {save_path}")
165
166                 if ui_image_callback and cfg.show_images:
167                     ui_image_callback(vis, f"Single Cross {i}")
168                 elif cfg.show_images:
169                     cv2.imshow("Single Cross", vis)
170                     cv2.waitKey(0)
171                     cv2.destroyAllWindows()
172                 continue
173
174         mask = binarizer.process(img)
175         raw_points = detector.detect_points(mask, gray)
176

```

```

177 pixel_points, _ = PerspectiveCorrector(
178     grid_size=(cfg.grid_rows, cfg.grid_cols),
179     actual_step_mm=cfg.marker_step_mm,
180 ).process_and_restore_grid(raw_points)
181 if pixel_points is None or len(pixel_points) == 0:
182     print(f"Не удалось восстановить точки для кадра {i}")
183     continue
184
185 circled_points = detect_circled_points(mask, pixel_points)
186 coordinate_systems = coord_detector.detect(
187     pixel_points,
188     circled_points,
189     grid_rows=cfg.grid_rows,
190     grid_cols=cfg.grid_cols,
191 )
192 dt = time.time() - start_time
193
194 if cfg.show_statistics:
195     print("-" * 50)
196     print(f"MODE: GRID")
197     print(f"raw points: {len(raw_points)}")
198     print(f"restored: {len(pixel_points)}")
199     print(f"systems: {len(coordinate_systems)}")
200     print(f"time: {dt:.4f} sec")
201     print("-" * 50)
202
203 if cfg.show_coordinate_systems:
204     for idx, s in enumerate(coordinate_systems):
205         print(f"\nSystem {idx}")
206         print("origin:", s["origin"])
207         print("x_axis:", s["x_axis"])
208         print("y_axis:", s["y_axis"])
209
210 if save_coords:
211     coords_filename = os.path.join(coords_dir, f"coordinates_{i}.csv
212 ")
213     save_coordinates_to_csv(coordinate_systems, pixel_points,
214 coords_filename, i)
215     print(f"Координаты" сохранены: {coords_filename}")
216
217     # Преобразуем точки в список для JSON сериализации
218     if isinstance(pixel_points, np.ndarray):
219         points_list = pixel_points.tolist()
220     else:
221         points_list = list(pixel_points)
222
223     all_results.append({
224         'image': i,
225         'pixel_points': points_list,
226         'coordinate_systems': coordinate_systems
227     })
228
229 if cfg.show_images:
230     res_img = visualizer.draw_detected_points(img, pixel_points)
231
232 if save_images:
233     save_path = os.path.join(images_dir, f"grid_{i}.jpg")
234     cv2.imwrite(save_path, res_img)
235     print(f"Изображение" сохранено: {save_path}")

```



```

235         if ui_image_callback:
236             ui_image_callback(res_img, f"Result {i}")
237         else:
238             cv2.imshow(f"Mask {i}", mask)
239             cv2.imshow(f"Result {i}", res_img)
240             cv2.waitKey(0)
241             cv2.destroyAllWindows()
242
243     if save_coords and all_results:
244         all_results_path = os.path.join(coords_dir, "all_results.json")
245         try:
246             with open(all_results_path, 'w', encoding='utf-8') as f:
247                 json.dump(all_results, f, indent=2, ensure_ascii=False,
default=str)
248                 print(fВсе" результаты сохранены в: {all_results_path}")
249         except Exception as e:
250             print(fОшибка" сохранения всех результатов: {e}")
251
252
253 def main():
254     args = parse_args()
255
256     cfg = AppConfig()
257
258     if args.config:
259         cfg = load_config_from_json(args.config)
260
261     if args.save_images:
262         cfg.save_images = True
263     else:
264         cfg.save_images = False
265
266     if args.save_coords:
267         cfg.save_coords = True
268     else:
269         cfg.save_coords = False
270
271     if args.config and not os.path.exists(args.config):
272         save_config_to_json(cfg, args.config)
273
274     if args.gui:
275         printЗапуск(" графического интерфейса...")
276         gui = AppGui(run_pipeline_callback=run_pipeline)
277         gui.mainloop()
278         return
279
280     ui = ConsoleInterface()
281     if not args.no_ui and not args.fast:
282         ui.run()
283     cfg = ui.config
284     if args.fast:
285         cfg.show_images = False
286         cfg.show_statistics = True
287         cfg.show_coordinate_systems = False
288         cfg.verbose = False
289
290     run_pipeline(
291         cfg,
292         save_images=args.save_images,
293         output_dir=args.output_dir,

```

```
294         save_coords=args.save_coords
295     )
296
297
298 if __name__ == "__main__":
299     main()
```

Листинг 9: main-файл